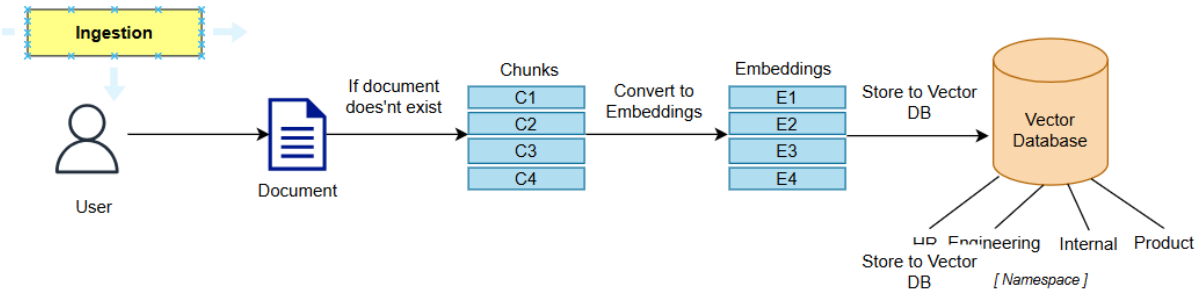
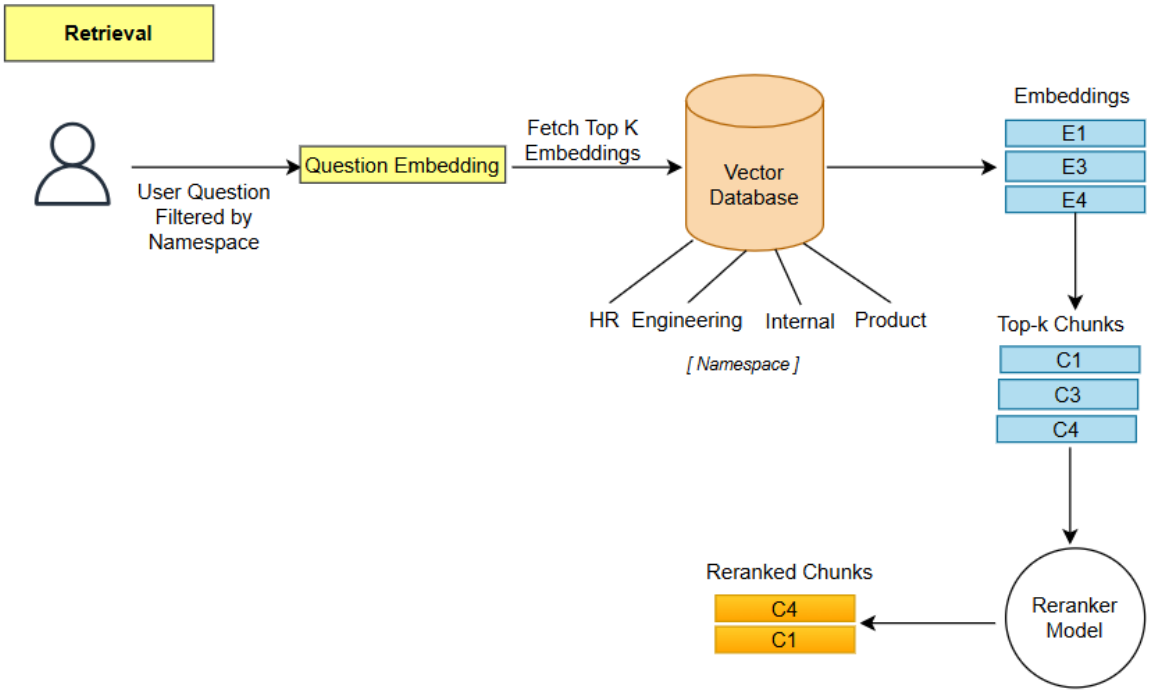
Retrieval Augmentation Generation (RAG)

Basics RAG Architecture:



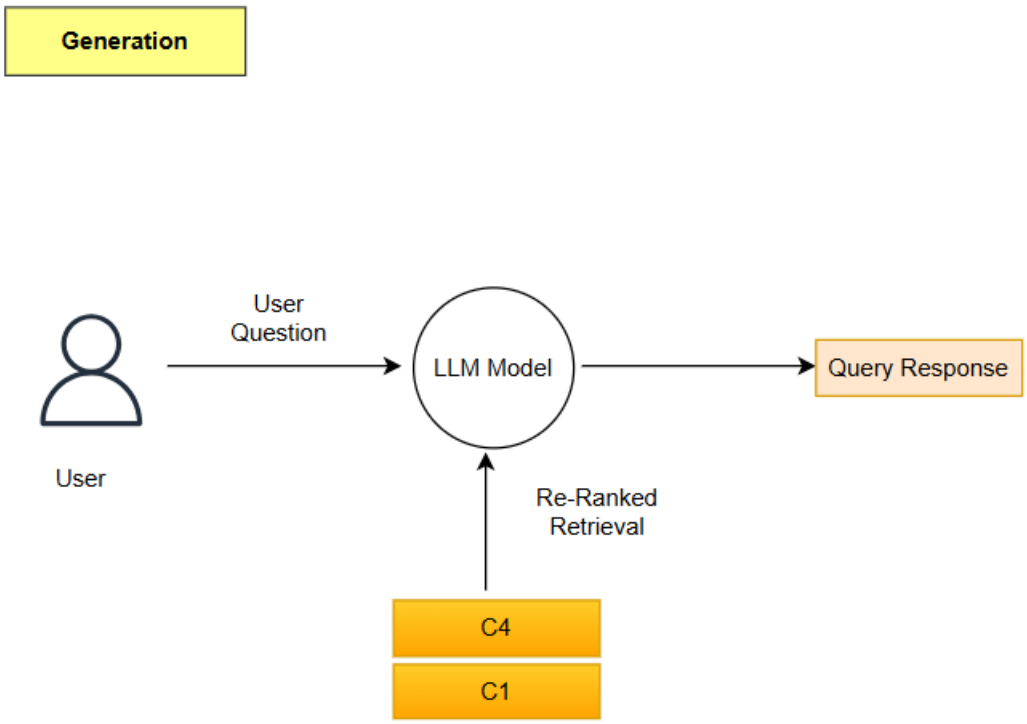
Data Ingestion: The goal is to transform raw data into a searchable format.

- Loading: Raw documents (PDFs, text, md etc.) are loaded using LangChain or similar tools.
- Chunking: Large texts are split into smaller, manageable chunks to fit model context windows.
- Embedding: Chunks are converted into numerical vectors using an embedding model like OpenAI.
- Indexing with Namespaces: Vectors are stored in a Vector Database like Pinecone, Chroma DB, FAISS etc. Namespaces are used to partition data by user, department or document type for isolated and secure searches.



Retrieval & Reranking: When a user asks a question, the system finds the most relevant information.

- Query Embedding: The user's query is converted into a vector using the same embedding model used during ingestion.
- Namespace Filtering: The search is restricted to a specific Namespace (e.g., only "HR documents") to ensure security and speed.
- Initial Retrieval: The system performs a similarity search to fetch the "Top-K" most similar chunks.
- Reranking: A specialized model (e.g., Cohere Rerank) re-scores these initial results for more precise semantic relevance, filtering out "noisy" or less relevant data before it hits the LLM.



Generation : The final step turns retrieved data into a human-like answer.

- Prompt Augmentation: The top reranked chunks are combined with the original user query into a single, enriched prompt.
- LLM Generation: A Large Language Model (e.g., GPT-4) uses this extra context to generate a factually grounded response.